

Schedule Importer AI – Technical & Implementation Plan

Backend-first architecture using FastAPI (Python)

1. Project Objective

The objective of this project is to build a multi-user web application that can ingest schedule images (screenshots or photos), extract structured calendar events using an AI model with vision capabilities, allow user review and correction, and then write the approved events into each user's own Google Calendar via OAuth-based authorization.

2. Core Capabilities

- Google OAuth login (each user connects their own Google account).
- Image upload for schedules (timetables, notes, screenshots).
- AI-based extraction of events into strict JSON structures.
- Human review and approval before calendar write.
- Insertion, update, and deletion of events in Google Calendar.
- Extensible to natural-language schedule edits.

3. High-Level Architecture

- Frontend: Web UI for upload, review, and confirmation.
- Backend: FastAPI server handling auth, AI calls, and Calendar API calls.
- AI Layer: Vision-capable LLM with structured JSON outputs.
- External APIs: Google Calendar API for event management.
- Database: Users, OAuth tokens, extraction jobs, event mappings.

4. Data Model (Minimum)

- Users: id, email, name, created_at.
- Google Tokens: user_id, access_token, refresh_token (encrypted), expiry.
- Extraction Jobs: image reference, parsed events JSON, timestamps.
- Calendar Event Links: mapping between internal events and Google eventId.

5. AI Output Contract

The AI must return events using a predefined JSON schema to ensure reliability. Each event includes title, start/end datetime (ISO 8601), timezone, optional recurrence, confidence score, and ambiguity notes. This enables deterministic backend processing.

6. API Endpoints (Illustrative)

- Auth: /auth/google/login, /auth/google/callback.

- Extraction: /extract/from-image.
- Calendar: /calendar/test-event, /calendar/events/batch-insert.
- Commit: /commit (approve reviewed events).

7. Implementation Milestones

- Milestone 1: Google OAuth + create a test calendar event.
- Milestone 2: Image upload + AI extraction (no calendar write).
- Milestone 3: Approve and insert extracted events.
- Milestone 4: Natural-language add/move/delete operations.

8. Security & Privacy Considerations

- API keys stored server-side only.
- Minimal Google OAuth scopes (least privilege).
- Encrypted storage of refresh tokens.
- Short retention for uploaded images unless user opts in.

9. Recommended Technology Stack

- Backend: FastAPI + Uvicorn.
- AI: OpenAI API (vision + structured outputs).
- Database: PostgreSQL (or SQLite for MVP).
- Deployment: Render, Fly.io, or Railway.